

Bridge Automation Framework - Developer Handover Guide

Last Updated: February 06, 2026

Author: Automation Team

1. Introduction

This repository contains the end-to-end test automation framework for the **NIOS Bridge Course Portal**. It covers critical workflows including **User Registration**, **Login/Logout**, **Dashboard Navigation**, **Payment Processing (SabPaisa)**, **Document Uploads**, and **Post-Login Services**.

The framework is built using **Python** and **Selenium WebDriver**, following the **Page Object Model (POM)** design pattern for maintainability and scalability.

2. Technology Stack

- **Language:** Python 3.9+
 - **Web Driver:** Selenium WebDriver
 - **Test Runner:** Pytest
 - **Reporting:** pytest-html
 - **Data Generation:** Faker
 - **Configuration:** ConfigParser (.ini files)
-

3. Repository Structure

The project maps the application pages to class files and separate test logic from page interaction logic.

```
Bridge-Automation.-Framework-/ ■■■■ config/ ■ ■■■■ config.ini # CENTRAL CONFIG:
URLs, Credentials, Timeouts, Payment Data ■■■■ pages/ # PAGE OBJECT MODEL (POM)
```

```
classes ■■■■ base_page.py # Parent class with wrapper methods (click, type, wait)
■■■■ registration_page.py # Registration Step 1 ■■■■ payment_flow_page.py #
SabPaisa Gateway Logic ■■■■ dashboard_page.py # Post-login Dashboard ■■■■ ...
(one file per page) ■■■■ tests/ # TEST SCRIPTS ■■■■ conftest.py # Pytest Fixtures
(Browser Setup/Teardown) ■■■■ test_registration.py # E2E Registration Flow ■■■■
test_login.py # Login/Logout Tests ■■■■ test_functional_*.py # Functional
workflows (Security, Teacher Flow) ■■■■ ... ■■■■ utilities/ ■■■■ data_utils.py #
Helper for dynamic data (Emails, PDF generation) ■■■■ custom_logger.py # Logging
setup ■■■■ read_properties.py # Config reader ■■■■ test_data/ # Dynamic artifacts
■■■■ email_counter.txt # Counters for unique email generation ■■■■ dummy.jpg #
Auto-generated for uploads ■■■■ dummy.pdf # Auto-generated for uploads ■■■■ logs/
# Execution logs ■■■■ reports/ # HTML Test Reports ■■■■ requirements.txt # Python
Dependencies
```

4. Setup & Installation

Prerequisites

1. Python 3.installed.
2. Chrome Browser installed.
3. ChromeDriver (managed automatically by Selenium Manager in newer versions, or ensure it's in PATH).

Installation Steps

1. **Clone the repository** (or unzip the project folder).
2. **Install Dependencies:**

```
bash pip install -r requirements.txt
```

5. Configuration (`config.ini`)

All environment-specific settings are in `config/config.ini`. **Do not hardcode values in scripts.**

Key	Description
-----	-------------

<code>[common_base_url_info]</code>	URL for the registration page.
<code>explicit_wait</code>	Global timeout for finding elements (default: 10s).
<code>[login_url]</code>	URL for the login page.
<code>email, password</code>	Credentials for login tests.
<code>[payment_card_number]</code>	Test card for SabPaisa gateway.
<code>[payment_email_counter_file]</code>	Tracks the incremental number for unique emails.
<code>[manual_otp_wait]</code>	Time allowed for Manual OTP entry (default: 120s).

6. Running Automations

The framework uses **Pytest**. You can run all tests or specific subsets.

A. Run All Tests

```
python -m pytest tests/
```

B. Run Specific Test Suite (e.g., Registration)

```
python -m pytest tests/test_registration.py -v
```

C. Run with HTML Report

```
python -m pytest tests/ --html=reports/report.html
```

7. Key Modules & Logic Explanation

A. Registration Flow & Unique Emails

- **Logic:** The system requires a unique email for every registration.
- **Solution:** `utilities/data_utils.py` reads `test_data/email_counter.txt`, increments the number, and generates emails like `insphere.shubhamsingh+105@gmail.com`.
- **Manual OTP:** The registration flow includes a Human-in-the-loop step. The script pauses for 120 seconds (configurable) for you to manually enter the OTP sent to the email/mobile.

B. Robust Dropdown Handling (Chosen.js)

The portal uses "Chosen.js" dropdowns which hide standard `<select>` tags.

* **Strategy:** The `BasePage` class has a smart method `select_chosen_option` that:

1. Clicks the visible container to open the dropdown.
2. Types the search text.
3. Presses ENTER.
4. *Fallback:* If UI interaction fails, it uses JavaScript to unhide the original `<select>` element and selects the value programmatically.

C. Payment Gateway (SabPaisa)

- **IFrame Handling:** The payment page often loads inside an `iframe`. The automation (`pages/payment_flow_page.py`) automatically detects and switches frames.
- **Details:** It enters the dummy card details defined in `config.ini` and simulates a successful transaction.

D. File Uploads

- **Dynamic Creation:** Tests do not rely on pre-existing files. `data_utils.py` creates `dummy.jpg` and `dummy.pdf` at runtime if they don't exist, ensuring tests are portable.

8. Maintenance Guide

How to Add a New Page

1. Create a new class in `pages/` (e.g., `HistoryPage`).
2. Inherit from `BasePage`.
3. Define locators as class tuples: `BUTTON = (By.ID, "btn_full_id")`.
4. Write methods for actions (e.g., `download_receipt`).

How to Fix "Element Not Found" Errors

1. Check if the element is inside an **iframe**.
2. Increase `explicit_wait` in `config.ini`.
3. Check if the locator (ID/XPath) has changed in the source code.

9. Troubleshooting

Issue: *Script hangs at OTP screen.*

- **Fix:** You must manually enter the OTP in the browser window within the timeout period (default 120s).

Issue: *Dropdown selection fails.*

- **Fix:** Ensure the text in the test data matches the dropdown text exactly (or check case sensitivity).

Issue: *Browser closes too fast.*

- **Fix:** Use `time.sleep()` in debugging or check the `teardown` method in `tests/conftest.py`.

10. Important File References

For more granular details, refer to these existing documents in the project:

* `TECHNICAL_DOCS.md`: Technical architecture deep-dive.

* `AUTOMATION_WALKTHROUGH.md`: Step-by-step logic map for the Registration script.